

ВВЕДЕНИЕ

Разработка дронов — это одна из самых живых инженерных тем сегодня. Дроны делают фотосъемку с воздуха, следят за природой, доставляют посылки, помогают спасателям и правоохранительным органам. Обществу нужны простые, надежные и недорогие системы управления для квадрокоптеров, чтобы их могли использовать и в коммерческих проектах, и в исследовательских. Главной задачей сейчас стоит создать алгоритмы, которые обеспечат стабильный и точный полет, даже если у дрона слабые вычислительные способности. Это и делает проект актуальным на данный момент времени.

Теоретическая значимость заключается в создании и улучшении алгоритмов управления, обеспечивающих стабильный полет дронов. С практической стороны в результате работы мы получаем прототип квадрокоптера с реализованным полетным контроллером, который может быть использован для дальнейших разработок или образовательных целей. Написанный алгоритм управления может быть адаптирован для других типов дронов или автономных систем, повышая их универсальность.

Уже продолжительное время тема управления беспилотными аппаратами является одной из самых актуальных в научной и инженерной практике. В работе Anderson и Moore (2017)[1] акцентируется важность быстрой и точной обработки данных с инерциальных сенсоров для получения стабильного полёта дрона. Кроме того, многие исследования в области открытых платформ управления дронами (например Smith et al., 2020 [2]) указывают на то, что доступные микроконтроллеры все же имеют возможность обеспечивать достаточную производительность при корректной оптимизации алгоритмов. С другой стороны вопроса, существующие библиотеки для работы с сенсорами, как например Adafruit Unified Sensor, зачастую требуют модификаций для исполнения конкретных задач, что подчеркивает необходимость разработки специализированных решений. В

данной работе основой было принято взять создание собственной реализации взаимодействия с сенсорами и алгоритмов управления, что отличает ее от существующих решений.

Целью работы является разработка и реализация алгоритма управления для беспилотного квадрокоптера, обеспечивающего стабильность и точность полёта с использованием доступной аппаратной платформы на базе микроконтроллера Arduino Nano с микропроцессором ATmega328P, обладающим достаточной вычислительной мощностью для выполнения задач управления и стабилизации. Для реализации поставленных задач был собран квадрокоптер с Х-образной архитектурой двигателей. Для определения положения дрона в пространстве применялись платы Adafruit 9DOF (с сенсорами LSM303 и L3GD20) и BMP085 (барометр). Для обеспечения надежной работы была разработана собственная программная библиотека взаимодействия с сенсорами по протоколу I2C, что позволило оптимизировать обработку данных. В рамках работы реализованы следующие компоненты управления: фильтрация данных с помощью комплементарного фильтра, автоматическая калибровка сенсоров, управление высотой по данным барометра и алгоритмы стабилизации полёта.

ГЛАВА 1. Сборка и механика дрона

Данный раздел описывает процесс проектирования и сборки механической части квадрокоптера, включая выбор материалов, компонентов и подходов к их интеграции, а также решения проблем, возникших в процессе реализации.

1. Основа конструкции

Для сборки квадрокоптера была выбрана карбоновая рама TBS Source One V5, очень популярная платформа среди этого класса дронов. Компоненты из углеродного волокна толщиной 4 мм используются для основных несущих деталей; 2 мм — для дополнительных деталей. Карбон обладает большой прочностью при малом весе, но, что самое важное, он помогает снизить нагрузку на двигатели, а значит, увеличить время полета. Установка компонентов облегчается модульной конструкцией и простой заменой при необходимости. Монтаж двигателей осуществлялся по X-образной компоновке, для достижения баланса между устойчивостью и маневренностью, так как именно при такой компоновке происходит равномерное распределение тяги по всем четырем осям. Схема представлена на рисунке 1.

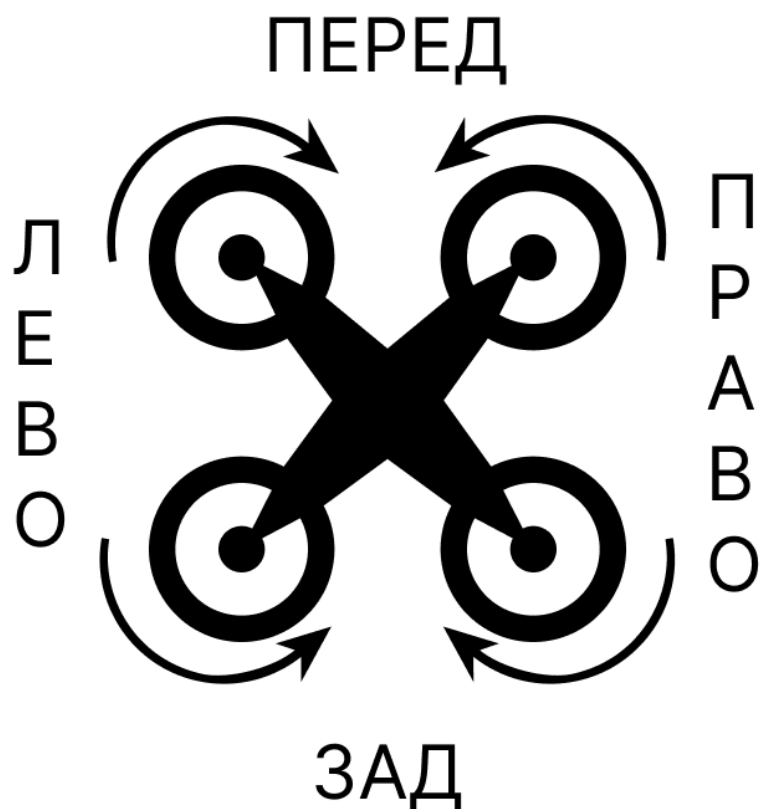


Рисунок 1 - Схема расположения двигателей

2. Двигатели

В качестве двигателей были выбраны четыре бесколлекторных модели XING 2207 с частотой вращения 2750 кВ, обладающие высокой эффективностью, что делает их отличным вариантом для дронов среднего размера. На каждый двигатель был установлен пропеллер размера 5x4.5x3 (5 дюймов в диаметре, шаг 4.5, трехлопастные).

Управление двигателями осуществляется с помощью электронных регуляторов скорости (ESC) модели BLHeli_S. Взаимодействие с этими регуляторами происходит посредством ШИМ сигнала, что сильно упрощает управление двигателями. Регулятор с одной стороны подключается к соответствующему двигателю, а с другой к аккумулятору и полетному контроллеру. В качестве простого и безопасного подключения питания использовались разъемы T-Plug, XT60, а также Dupont коннекторы для

подключения к плате микроконтроллера. Регуляторы были размещены во внутренней части дрона, чтобы минимизировать длину проводов и улучшить качество связи. Схема подключения регуляторов и моторов представлена на рисунке 2.

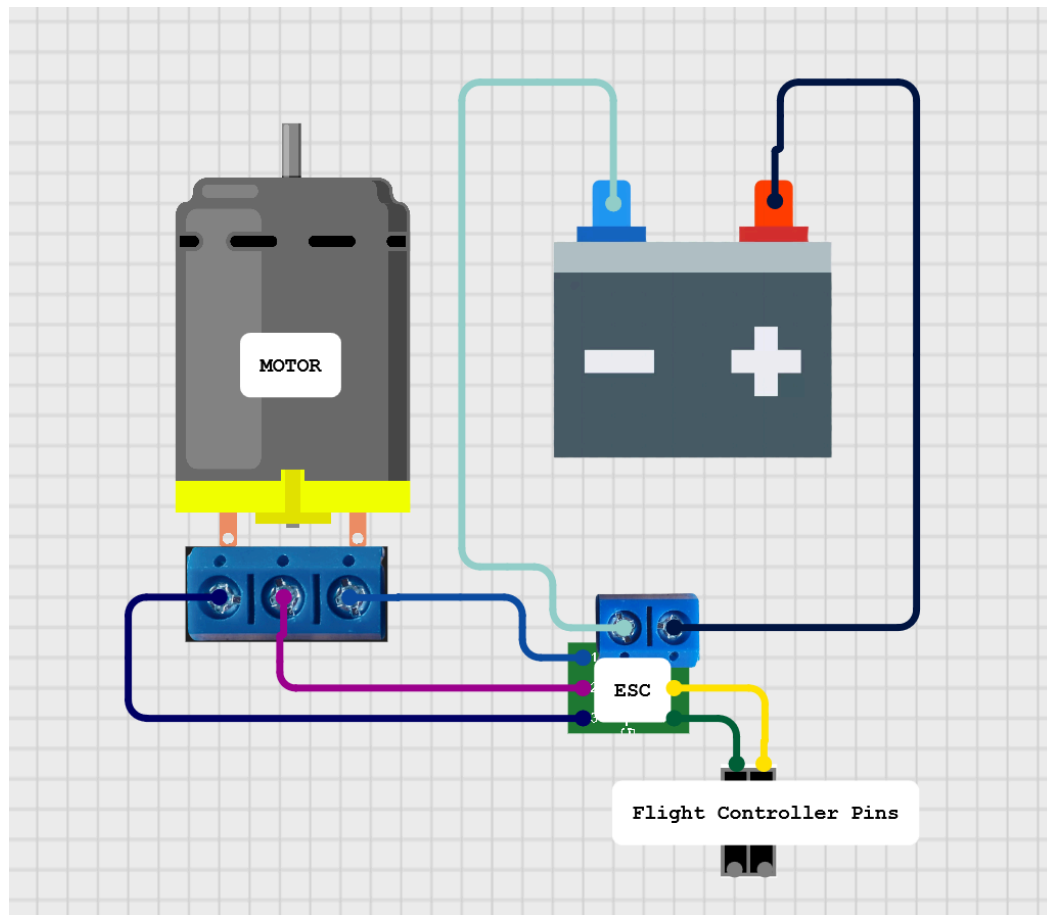


Рисунок 2 - Схема подключения моторов и регуляторов

3. Питание

Питание квадрокоптера осуществляется с помощью литий-полимерного аккумулятора с конфигурацией 3S (три ячейки, номинальное напряжение 11.1 В), емкостью 5200 мА·ч и максимальным током разряда 90С. Такой аккумулятор обеспечивает достаточную мощность для работы двигателей и электроники в течение 10–15 минут полета в зависимости от интенсивности маневров и внешних условий, таких как температура воздуха и ветер. Аккумулятор закреплен на раме с помощью регулируемых ремешков-липучек, что позволяет быстро заменить батарею и

обеспечивает надежную фиксацию во время полета. Для защиты аккумулятора от механических повреждений и вибраций были использованы комплектные амортизирующие прокладки.

С целью обеспечения питанием платы Arduino Nano использовался понижающий преобразователь напряжения (buck-конвертер) с выходом 7 В и током до 3 А. Конвертер обеспечивает стабильное питание контроллера и подключенных к нему датчиков, предотвращая сбои в процессе работы.

4. Контроллер и сенсоры

Центральным элементом системы управления является микроконтроллер Arduino Nano с микропроцессором ATmega328P, рабочая частота которого равна 16 МГц. Этот микроконтроллер был выбран благодаря его компактности, доступности и достаточной вычислительной мощности для реализации алгоритмов управления и обработки данных с датчиков.

Для определения пространственной ориентации квадрокоптера применялась плата Adafruit 9DOF, включающая датчики LSM303 (акселерометр и магнитометр) и L3GD20 (гироскоп). Акселерометр измеряет ускорение по трем осям, магнитометр определяет направление относительно магнитного поля Земли, а гироскоп фиксирует угловые скорости. С помощью этих данных будет происходить расчет углов ориентации коптера. Для расчета давления и высоты использовался барометрический датчик BMP085, который определяет атмосферное давление и температуру, а затем рассчитывает на их основе высоту с точностью до 0.25 м. Все датчики подключены к Arduino Nano через протокол I2C, который позволяет одновременно считывать данные с нескольких устройств, используя всего четыре провода для соединения (VIN, GND, SCL, SDA).

5. Управление

Для беспроводного управления квадрокоптером был использован радиомодуль NRF24L01 2,4 ГГц. Он обеспечивает односкачковую

двустороннюю надежную связь между удаленным устройством и квадрокоптером в пределах 1000 м в условиях прямой видимости. Задержка передачи данных очень низкая для оперативного управления, что идеально подходит для требований дрона. Этот модуль подключен к Arduino Nano через интерфейс SPI, а внешняя антенна используется для повышения стабильности соединения. На удаленной стороне был подключен аналогичный модуль NRF24L01, джойстики и потенциометры, которые используются для отправки команд

Схема подключения платы Arduino Nano, сенсоров, радиомодуля и регуляторов оборотов представлена на рисунке 3.

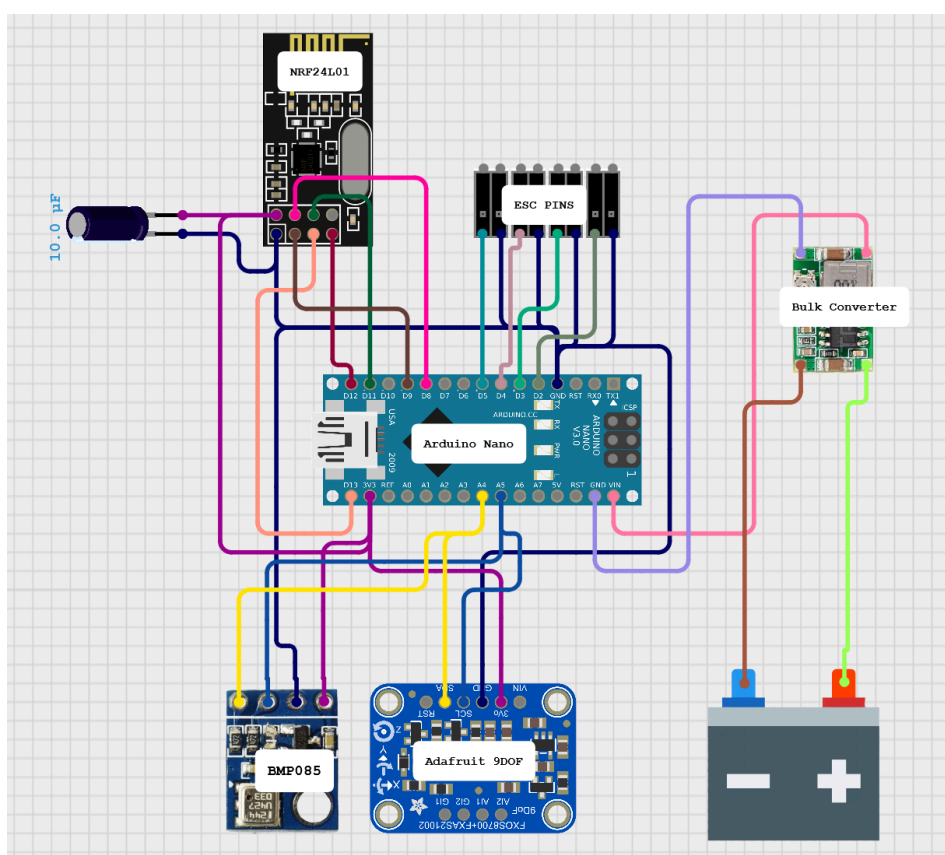


Рисунок 3 - Схема основных компонентов

6. Процесс сборки

В первую очередь на раму были установлены двигатели, после чего была проведена проверка их ориентации для корректного направления тяги. После это на каркас были поставлены регуляторы скорости, которые

подключались к двигателям и аккумулятору через самодельную плату распределительного питания. Кроме того, на распределительной плате так же размещался понижающий преобразователь напряжения. Вся конструкция в свою очередь была закреплена во внутренней части корпуса. Макетная плата с припаянной Arduino Nano, всеми датчиками и радиомодулем NRF24L01 закреплялась поверх аккумулятора на гибких связках для обеспечения легкого доступа к серийному порту Arduino. Для фиксации проводов были использованы нейлоновые стяжки для предотвращения их смещения во время полета.

В процессе сборки возникли несколько технических сложностей. Основной проблемой стало расположение регуляторов скорости, которые имеют свойство считывания помех при некорректной установке. Для решения этой проблемы регуляторы были перенесены с ответвлений в центр корпуса, а также были скручены сигнальные провода, идущие до Arduino Nano. Еще одной проблемой стало некорректная работа регуляторов скорости в некоторых случаях. Для устранения был проведен процесс модификации прошивки регуляторов с помощью Arduino Uno и программного обеспечения BIHeliSuite. Кроме того при использовании радиомодуля были замечены помехи, приводящие к некорректным данным. Проблема заключалась в высокой чувствительности модуля к входному напряжению. Для решения непосредственно на радиомодуль был установлен конденсатор емкостью 10 микрофарад.

Итогом сборки стал квадрокоптер с полной массой около 700 г (включая аккумулятор), обладающий высокой прочностью и устойчивостью. Конструкция получилась компактной, с оптимизированным расположением компонентов, что упростило дальнейшую настройку программного обеспечения и тестирование алгоритмов управления. Созданная механическая основа обеспечивает надежную платформу для реализации задач управления и стабилизации, описанных в последующих разделах работы.

ГЛАВА 2. Работа с датчиками

В качестве основы для работы с датчиками изначально был использован пакет библиотек разработки Adafruit. В ходе выполнения работ по созданию полетного контроллера была замечена проблема сильно ограниченной памяти микроконтроллера Nano. В этих условиях использование сторонних библиотек с большим количеством излишнего функционала оказалось под вопросом. После продолжительных тестов было принято решение отказаться от использования сторонних библиотек для работы с сенсорами, взамен которым необходимо было создать свой функционал, более разумно использующий ресурсы микроконтроллера.

Как уже упоминалось, для работы с чипами сенсоров используется протокол I2C, предлагающий адресную систему взаимодействия между устройствами, используя при этом всего 4 провода [3]. Принцип работы I2C основан на схеме, которая включает в себя:

- SDA (Serial Data Line) — линия данных, по которой передаются данные;
- SCL (Serial Clock Line) — линия тактовых импульсов, которая синхронизирует передачу данных.

В схеме может быть только один мастер(ведущий), а все остальные подчиненные(ведомые). Мастер инициирует передачу данных, генерируя тактовые импульсы и управляющие сигналы, в то время как подчиненные отвечают на запросы мастера. Схема подключения представлена на рисунке 4.

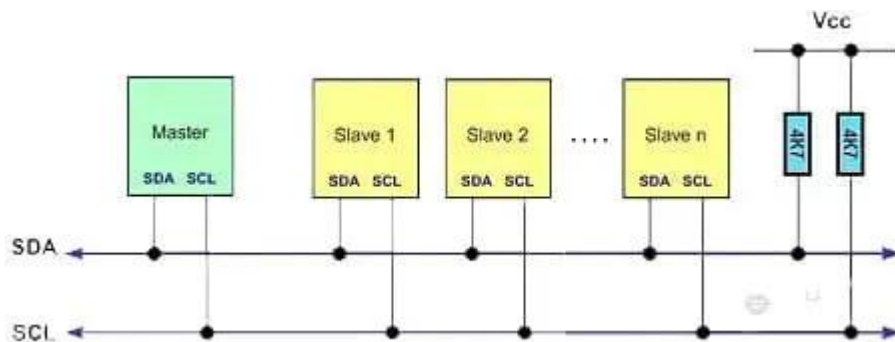


Рисунок 4 - Схема подключения устройств через интерфейс I2C

В силу распространенности протокола, современные платы Arduino (включая Nano v3) поставляются с предустановленной библиотекой Wire.h, которая предоставляет готовый интерфейс для быстрого и простого создания функционала взаимодействия с I2C устройствами.

Для создания библиотеки была использована документация проектов Adafruit Unified L3GD20 Driver[4] и Adafruit LSM303 Accelerometer/Magnetometer Library[5]. Кроме того, для понятия принципов работы сенсоров и данных о структуре регистров была использована соответствующая документация, опубликованная в открытый доступ компанией STMicroelectronics[6].

В ходе написания функционала считывания данных акселерометра, гироскопа, магнетомера и барометра, были также написаны функции для упрощенной работы с регистрами датчиков. Так например, для активации и конфигурации датчиков была использована функция writeRegister (листинг 1), которая реализовала функционал записи данных в регистры чипов сенсоров.

```
void writeRegister(uint8_t addr, uint8_t reg, uint8_t
value) {
    Wire.beginTransaction(addr);
    Wire.write(reg);
    Wire.write(value);
    Wire.endTransmission();
}
```

Листинг 1 - Код функции writeRegister

А для считывания данных из регистров была написана функция readMultiple (листинг 2).

```
void readMultiple(uint8_t addr, uint8_t reg, uint8_t len,
uint8_t *dest) {
    Wire.beginTransaction(addr);
    Wire.write(reg);
    Wire.endTransmission(false);
    Wire.requestFrom(addr, len);
    for (uint8_t i = 0; i < len && Wire.available(); i++) {
        dest[i] = Wire.read();
    }
}
```

Листинг 2 - Код функции readMultiple

Для конфигурации и включения датчиков используются соответствующие регистры датчиков. Ключевые конфигурационные регистры датчиков включают в себя CTRL_REG4 гироскопа (таблица 1), а также часть регистра CRB_REG_M магнитометра.

Таблица 1 - Структура регистра CTRL_REG4

BDU	BLE	FS1	FS2	-	0 ⁽¹⁾	0 ⁽¹⁾	SIM
Block data update.	Big/little endian data selection.	Full scale selection					SPI serial interface mode selection

Так, к примеру, заполняя регистр CTRL_REG4 значением 0x00 мы устанавливаем максимальную чувствительность гироскопа в значение 250 dps (degrees per second).

В ходе написания функционала по вычислению высоты дрона на основе данных с датчика BMP085 возникла проблема появления продолжительных задержек, связанных с принципом работы сенсора. В худших случаях время выполнения одного цикла считываний доходило до 22 мс, что является превышением теоретического времени цикла более чем в два раза. Для разрешения этой проблемы была использована так называемая “Машина Состояний”, которая позволяет избежать продолжительного простоя контроллера, используя факт того, что изменение высоты не является таким же резким, как и изменения углов ориентации. Таким образом, при вызове функции считывания высоты контроллер проходит по одному из этапов и переводит “Машину” в следующее состояние. Всего таких состояний три: IDLE, WAIT_TEMP и WAIT_PRESS. На этапе IDLE производится запрос на актуальную температуру, после чего выставляется состояние WAIT_TEMP. Во время работы этапа WAIT_TEMP контроллер

запрашивает данные об актуальном давлении с сенсора и выставляет состояние WAIT_PRESS. На последнем этапе происходит вычисление актуальной высоты, в конце которого выполняется сброс состояния на этап IDLE.

В результате работы была получена библиотека, реализующая систему неблокирующих процессов, время выполнения которых (~ 6 мс) входит в диапазон теоретической продолжительности цикла считываний (< 10 мс).

ГЛАВА 3. Комплементарный фильтр

Для определения углов наклона объекта в пространстве в аэродинамике и робототехнике используется система из трех углов:

- Roll (крен) отвечает за вращение объекта вокруг его продольной оси, проходящей через центр массы;
- Pitch (тангаж) - это вращение объекта вокруг его поперечной оси, также проходящей через центр массы;
- Yaw (рыскание) - это вращение объекта вокруг его вертикальной оси, также проходящей через центр массы.

Представление базируется на углах Эйлера, описывающие поворот абсолютно твердого тела в трехмерном евклидовом пространстве.

Точные значения положения дрона в пространстве критически важны для корректной работы алгоритмов управления. Для вычисления углов наклона из данных акселерометра, гироскопа и магнитометра необходим так называемый Комплементарный фильтр, задача которого комбинировать и преобразовывать данные с нескольких датчиков и выводить углы наклона коптера [7]. Фильтр играет ключевую роль в работе дрона, так как он позволяет компенсировать недостатки работы датчиков по отдельности и выдавать точное физическое представление положения в реальном времени [8]. В этом разделе описывается принцип работы фильтра, его реализация и сравнение с альтернативными решениями.

1. Теоретическая основа углов ориентации

Углы ориентации дрона определяются на основе данных инерциальных датчиков: акселерометра, гироскопа и магнитометра. Акселерометр измеряет ускорение датчика по трем осям (x , y , z), включая гравитационное ускорение, что позволяет вычислить углы крена и тангажа относительно вектора силы тяжести. Формулы для вычисления углов крена (φ) и тангажа (θ) на основе данных акселерометра (a_x , a_y , a_z) имеют следующий вид:

$$\phi = \arctan 2(a_y, a_z),$$

$$\theta = \arctan 2(-a_x, \sqrt{a_y^2 + a_z^2}),$$

где $\arctan 2$ является функция арктангенса, учитывающая знаки аргументов. Значения, вычисленные данным способом, являются очень точными при сохранении статичного положения и движении с постоянной скоростью. Однако же, при наличии ускорений, данные акселерометра подаются с большим количеством шумов, что не позволяет использовать только их.

Гироскоп измеряет угловые скорости по трем осям $(\omega_x, \omega_y, \omega_z)$, которые можно интегрировать по времени для получения углов ориентации:

$$\phi(t) = \phi(t_0) + \int_{t_0}^t \omega_x(\tau) d\tau,$$

$$\theta(t) = \theta(t_0) + \int_{t_0}^t \omega_y(\tau) d\tau,$$

$$\psi(t) = \psi(t_0) + \int_{t_0}^t \omega_z(\tau) d\tau.$$

Минусом использования гироскопа является интегральная природа вычислений, что приводит к накопительной ошибке (дрейфу), появляющейся благодаря наличию шумов в данных. Это делает показания ненадежными при продолжительном использовании.

Принцип работы магнитометра основывается на измерении направления магнитного поля Земли. Датчик позволяет корректировать угол Рыскания, но его работа сильно чувствительна к внешним магнитным помехам (например мощным двигателям).

Для получения точных углов наклона дрона требуется комбинирование данных, полученных с нескольких датчиков. Это позволяют сделать

алгоритмы слияния данных (sensor fusion), наиболее популярным из которых является комплементарный фильтр.

2. Принцип работы комплементарного фильтра

Комплементарный фильтр основан на идее слияния данных акселерометра и гироскопа. Кроме этого в различных вариациях возможно также использование магнитометра, но в данной реализации он не задействуется. Как уже было замечено, акселерометр передает довольно точные данные о крене и тангаже в статических или медленно меняющихся условиях, но подвержен шумам на высоких частотах из-за вибраций и ускорений. Гироскоп, наоборот, точен на высоких частотах, где он хорошо отслеживает быстрые изменения ориентации, но на низких частотах страдает от дрейфа из-за накопления ошибок при интегрировании.

В этой реализации комплементарный фильтр комбинирует данные с помощью взвешенной суммы:

$$\phi = \alpha * \phi_{gyro} + (1 - \alpha) * \phi_{accel}$$

$$\theta = \alpha * \theta_{gyro} + (1 - \alpha) * \theta_{accel}$$

где α — коэффициент “доверия” фильтра к гироскопу (обычно в диапазоне 0.9–0.99), ϕ_{gyro} и θ_{gyro} — углы, полученные интегрированием данных гироскопа, а ϕ_{accel} и θ_{accel} — углы, вычисленные по данным акселерометра. Высокое значение α (например, 0.98) предпочтительно для квадрокоптеров, так как гироскоп обеспечивает точность при быстрых маневрах, а акселерометр корректирует дрейф на длительных интервалах.

3. Реализация комплементарного фильтра

Алгоритм работает следующим образом:

1) Сбор данных с датчиков. С акселерометра запрашиваются ускорения по осям x , y , z , а с гироскопа — угловые скорости ω_x , ω_y , ω_z в

градусах в секунду. Угловые скорости затем преобразуются в радианы в секунду умножением на коэффициент 0.0174533 ($\pi/180$);

2) Расчет времени. Для интегрирования угловых скоростей, полученных с гироскопа, определяется временной интервал dt между текущим и предыдущим измерением с использованием функции `micros()` для получения времени в микросекундах;

3) Вычисление углов акселерометра. Углы крена и тангажа рассчитываются по приведенным выше формулам с использованием функции `arctan 2`. Угол рысканья не вычисляется из данных акселерометра, так как для этого требуется магнитометр, который в данной реализации не используется из-за помех;

4) Интегрирование данных гироскопа. Углы ориентации (ϕ, θ, ψ) обновляются путем интегрирования угловых скоростей:

$$\phi += \omega_x d\tau$$

$$\theta += \omega_y d\tau$$

$$\psi = \omega_z d\tau$$

5) Применение комплементарного фильтра. Углы крена и тангажа корректируются с использованием комплементарного фильтра:

$$\phi = \alpha * \phi + (1 - \alpha) * \phi_{accel}$$

$$\theta = \alpha * \theta + (1 - \alpha) * \theta_{accel}$$

где $\alpha=0.98$, что представляет данным гироскопа приоритет, но позволяет акселерометру корректировать дрейф

6) Преобразование в градусы. Для вывода и дальнейшего использования углы переводятся из радиан в градусы умножением на $180/\pi$.

7) Частота обновления. Алгоритм выполняется с частотой около 100 Гц, что достигается добавлением задержки в 10 мс (`delay(10)`), обеспечивая достаточную частоту для управления квадрокоптером в реальном времени.

Код оптимизирован для минимального времени вычислений, избегая использования усложненных библиотек.

4. Сравнение с альтернативными методами

Изначально в проекте рассматривалось использование готовых решений в области слияния данных с датчиков, но каждое из рассмотренных решений имело свой ряд неудобств по сравнению с собственной имплементации Комплементарного фильтра. Из опробованных методов можно выделить такие фильтры как Madgwick и Mahony, которые часто используются в системах управления беспилотниками. Рассмотрим их особенности и сравним с собственным решением.

Фильтр Madgwick - это алгоритм, предложенный Себастьяном Мадгвиком, использующий в качестве функции минимизации ошибки ориентации градиентный спуск, комбинируя данные акселерометра, гироскопа и магнитометра. Он работает путем вычисления ориентации в виде кватернионов, что позволяет избежать проблем с сингулярностью (gimbal lock), характерной для углов Эйлера. Этот фильтр эффективен для систем с высокой динамикой и точными датчиками, но требует значительных вычислительных ресурсов из-за итеративных вычислений и работы с кватернионами. Для выбранной Arduino Nano с ограниченной производительностью (16 МГц) и памятью это приводит к большим задержкам, что критично для работы дрона.

Алгоритм Mahony также работает с кватернионами и использует пропорционально-интегральный регулятор для коррекции ошибок на основе данных акселерометра и магнитометра. Он менее требователен к вычислительным ресурсам, но все же требует вычисления матриц вращения и нормализации кватернионов. Это сильно увеличивает время обработки данных, что также непозволительно для корректной работы дрона.

Несмотря на значительные преимущества, используемый комплементарный фильтр имеет свой ряд ограничений. Высокая

чувствительность к качеству калибровки датчиков приводит к ошибкам в начальных данных акселерометра и гироскопа, которые могут привести к неточности вычисляемой ориентации. Кроме того, фильтр не использует данные магнитометра для коррекции рысканья, что является проблемой при длительных полетах, где дрейф гироскопа становится заметным. В данной работе эта проблема компенсируется регулярной калибровкой датчиков перед полетом.

Для улучшения точности в будущем можно рассмотреть гибридный подход, где комплементарный фильтр комбинируется с элементами фильтра Mahony для повышения точности при минимальном увеличении вычислительной нагрузки. Однако для текущей реализации комплементарный фильтр оказался оптимальным решением, обеспечивая баланс между производительностью, точностью и простотой.

ГЛАВА 4. Управление ориентацией дрона

Для управления положением дрона в работе применяется пропорционально-интегральный-дифференциальный регулятор (ПИД). Он выполняет задачу обработки данных, полученных с помощью комплементарного фильтра, и распределения тяги между двигателями для достижения требуемых углов ориентации. Раздел описывает теоретические основы управления положением дрона, принципы работы ПИД регулятора и его калибровку.

1. Основы управления ориентацией

Как уже было сказано, положение коптера в пространстве определяется с помощью трех углов, соответствующих ортогональной системе дрона. Для управления этими углами необходимо менять скорость вращения каждого из четырех двигателей, чтобы создать необходимые моменты силы.

Квадрокоптер с Х-образной архитектурой имеет четыре двигателя, обозначенные как М1 (передний левый), М2 (передний правый), М3 (задний правый) и М4 (задний левый). Управление ориентацией достигается за счет дифференциального изменения тяги двигателей:

- Крен (roll): Для наклона вправо увеличивают тягу двигателей М1 и М4 (левый борт) и уменьшают тягу М2 и М3 (правый борт). Для наклона влево делают наоборот.
- Тангаж (pitch): Для наклона вперед увеличивают тягу двигателей М3 и М4 (задние) и уменьшают тягу М1 и М2 (передние). Для наклона назад — наоборот.
- Рысканье (yaw): Для поворота по часовой стрелке увеличивают тягу двигателей М1 и М3 (диагональ с вращением против часовой стрелки) и уменьшают тягу М2 и М4. Для поворота против часовой стрелки делают наоборот.

Суммарная тяга, создаваемая двигателями, в свою очередь влияет на высоту дрона. В данном разделе основное внимание уделяется управлению ориентацией, так как работа с ней предстает наиболее сложной.

2. Принципы распределения тяги

Распределение тяги между двигателями осуществляется применением управляющих сигналов, вычисленных ПИД-регулятором. Как уже было сказано, каждый двигатель управляется через соответствующий электронный регулятор скорости (ESC) с использованием широтно-импульсной модуляции (ШИМ). Сигнал ШИМ задает уровень тяги в диапазоне от 0 до 100% (в данном случае используется конфигурация с диапазоном от 1200 до 1900 микросекунд). Для управления ориентацией формируются корректирующие сигналы для каждого двигателя на основе разницы между текущими и целевыми углами ориентации (ошибка управления).

Алгоритм распределения тяги представляется следующим образом. Пусть T_{base} — базовая тяга, обеспечивающая поддержание высоты, а ΔT_{roll} , ΔT_{pitch} , ΔT_{yaw} — корректирующие сигналы для управления креном, тангажом и рысканьем соответственно [9]. Тогда тяга для каждого двигателя рассчитывается как:

$$T_{M1} = T_{base} + \Delta T_{roll} + \Delta T_{pitch} + \Delta T_{yaw}$$

$$T_{M2} = T_{base} - \Delta T_{roll} + \Delta T_{pitch} - \Delta T_{yaw}$$

$$T_{M3} = T_{base} - \Delta T_{roll} - \Delta T_{pitch} + \Delta T_{yaw}$$

$$T_{M4} = T_{base} + \Delta T_{roll} - \Delta T_{pitch} - \Delta T_{yaw}$$

Значения ΔT_{roll} , ΔT_{pitch} , ΔT_{yaw} вычисляются ПИД-регулятором на основе ошибок по каждому из углов. Для предотвращения превышения допустимых значений тяги (например, отрицательной тяги или превышения максимума ESC) сигналы ограничиваются в известном диапазоне для входных сигналов (1200:1900 us).

3. Реализация ПИД-регулятора

ПИД-регулятор необходим для минимизации ошибки между текущими углами ориентации (ϕ, θ, ψ) , полученными с помощью комплементарного фильтра, и целевыми углами $(\phi_{target}, \theta_{target}, \psi_{target})$. ПИД-регулятор вычисляет управляющий сигнал $u(t)$ по формуле:

$$u(t) = K_p * e(t) + K_i * \int_0^t e(\tau) d\tau + K_d * \frac{de(t)}{dt},$$

где:

- $e(t)$ — ошибка $(\phi - \phi_{target}, \theta - \theta_{target}, \psi - \psi_{target})$,
- K_p — пропорциональный коэффициент, отвечающий за реакцию на текущую ошибку,
- K_i — интегральный коэффициент, устраняющий накопленную ошибку,
- K_d — дифференциальный коэффициент, учитывающий скорость изменения ошибки.

Для каждого угла ориентации (крен, тангаж, рысканье) используется свой ПИД регулятор. Алгоритм включает следующие шаги:

- 1) Сбор данных об ориентации. Текущие углы (ϕ, θ, ψ) получаются от комплементарного фильтра, описанного в предыдущем разделе.
- 2) Вычисление ошибки. Для каждого угла рассчитывается ошибка:

$$e_{roll}(t) = \phi - \phi_{target}$$

$$e_{pitch}(t) = \theta - \theta_{target}$$

$$e_{yaw}(t) = \psi - \psi_{target}$$

3) Обновление ПИД-регулятора. Для каждого угла вычисляется управляющий сигнал:

$$u(t) = K_p * e(t) + K_i * \sum e(\tau) d\tau + K_d * \frac{e(t) - e(t-1)}{dt}.$$

Интегральная составляющая представляется с использованием дискретной суммы, а дифференциальная — через разность ошибок за два последовательных шага.

4) Распределение тяги. Управляющие сигналы $u_{roll}, u_{pitch}, u_{yaw}$ преобразуются в корректирующие значения тяги $\Delta T_{roll}, \Delta T_{pitch}, \Delta T_{yaw}$ с учетом масштабирования.

5) Ограничение сигналов. Для предотвращения некорректных значений (например, отрицательной тяги) сигналы ограничиваются:

$$T_{M_i} = \max(1100, \min(1900, T_{M_i}))$$

6) Отправка сигналов. Вычисленные значения тяги передаются на ESC через ШИМ-выходы Arduino Nano.

4. Настройка ПИД-регулятора

Калибровка коэффициентов K_p, K_i, K_d проводилась экспериментально с использованием специально сконструированных тестовых стендов, так как точное моделирование динамики дрона практически невозможно с учетом ограниченности вычислительных ресурсов. Процесс настройки включал следующие этапы:

1) Начальная настройка K_p : Пропорциональный коэффициент подбирался первым, чтобы добиться быстрой реакции на ошибку без

сильных колебаний. Для крена и тангажа начальное значение $K_p = 9$ показало стабильное поведение, а для рысканья — $K_p = 8.5$ из-за меньшей инерции по этой оси.

2) Добавление K_d : Дифференциальный коэффициент ($K_p = 0.3$ для крена и тангажа, $K_p = 0.25$ для рысканья) был введен для гашения колебаний, возникающих при быстрых изменениях ориентации.

3) Интегральная составляющая K_i : Интегральный коэффициент ($K_p = 0.03$ для всех осей) добавлен для устранения стационарной ошибки, но его значение оставлено минимальным, чтобы избежать накопления ошибок при длительном использовании.

Приведенные значения обеспечили устойчивую стабилизацию при маневрах и удержании заданной ориентации.

ГЛАВА 5. Радиоуправление и связь

Стандартом для организации оперативной связи между квадрокоптером и пультом управления является использование радиомодуля. Радиоуправление обеспечивает беспроводную передачу команд управления от оператора к дрону, включая управление ориентацией, тягой и выполнением специальных маневров, таких как взлет и посадка, а также получение данных о состоянии дрона в реальном времени. В проекте для реализации двусторонней связи использован радиомодуль NRF24L01, работающий на частоте 2.4 ГГц. Пульт управления базируется на плате Arduino Micro. В этом разделе описываются принципы организации радиоуправления, аппаратное и программное обеспечение пульта, алгоритмы передачи и приема данных.

1. Принципы радиоуправления

Радиоуправление дроном предполагает передачу управляющих сигналов от пульта к дрону и получение телеметрии от дрона к пульту [10]. Управляющие сигналы включают:

- Уровень тяги (gas), определяющий общую мощность двигателей;
- Значения положения джойстика (X и Y), задающие целевые углы крена и тангажа;
- Специальные команды, такие как взлет (DEPART), посадка (LAND) или экстренное выключение (SHUTDOWN).

Критериями надежной связи являются минимальная задержка передачи данных, высокая помехозащищенность и достаточная дальность работы. В качестве основы радиосвязи был выбран радиомодуль NRF24L01, который поддерживает двустороннюю связь с динамической длиной пакетов, что позволяет не только передавать команды управления, но и телеметрию с дрона. Рабочая частота модуля равна 2.4 ГГц, дальность связи обеспечивается до 1000 м в условиях прямой видимости при использовании внешней антенны.

2. Аппаратное обеспечение пульта управления

Пульт управления построен на базе микроконтроллера Arduino Micro с процессором ATmega8P, работающим на частоте 16 МГц. Выбор данной платформы обусловлен ее компактностью, дешевизной и доступностью. Аппаратная часть пульта включает следующие компоненты:

1) Радиомодуль NRF24L01. Модуль подключен к Arduino Nano через интерфейс SPI (Serial Peripheral Interface) с использованием пинов CE (Chip Enable) и CSN (Chip Select Not). NRF24L01 может обеспечивать двустороннюю связь с дроном, поддерживая передачу данных с низкой задержкой (менее 1 мс). Для повышения дальности и устойчивости связи использовалась модель с внешней антенной. В связи с использованием дрона в научно-исследовательских целях был установлен низкий уровень мощности передачи (RF24_PA_LOW) для минимизировать энергопотребления.

2) Джойстик. Двухосевой аналоговый джойстик подключен к аналоговым пинам Arduino Nano (VRX_PIN для оси X, VRY_PIN для оси Y). Джойстик выдает аналоговые сигналы в диапазоне 0–1023, которые преобразуются в значения от -100 до 100 для задания углов крена и тангажа.

3) Поворотный энкодер. Энкодер используется для регулировки уровня тяги (gas). Он подключен к цифровым пинам (CLK_PIN и DT_PIN) с внутренними подтягивающими резисторами (INPUT_PULLUP). Энкодер генерирует сигналы при вращении, которые интерпретируются как увеличение или уменьшение тяги в диапазоне от -100 до 100. Это позволяет точно контролировать мощность двигателей.

4) Кнопки. Две кнопки (UP_PIN и DOWN_PIN) с подтягивающими резисторами используются для отправки специальных команд: взлет (DEPART) при нажатии верхней кнопки и посадка (LAND) при нажатии нижней.

5) Источник питания. Пульт питается от внешнего аккумулятора. Это позволяет сделать пульт портативным и автономным.

Аппаратные компоненты размещены на компактной плате, а провода зафиксированы для минимизации помех и повышения надежности.

3. Программное обеспечение пульта управления

Программное обеспечение пульта управления разработано с использованием библиотеки RF24. Программа выполняет следующие функции:

- 1) Инициализация радиомодуля, джойстика, энкодера и кнопок.
- 2) Считывание данных с устройств ввода (джойстик, энкодер, кнопки).
- 3) Формирование и отправка управляющих данных дрону.
- 4) Прием и обработка телеметрии от дрона.
- 5) Вывод диагностической информации через последовательный порт.

Основные структуры данных, используемые в программе:

– `Control_Data_t`: Содержит данные для управления дроном, включая положение джойстика (X, Y), уровень тяги (gas) и специальную команду (NONE_COMMAND, DEPART, LAND, SHUTDOWN).

– `State_t`: Описывает состояние дрона, включая текущую тягу (gas) и текущую команду (IDLE, DEPARTING, FLYING, LANDING, SHUTDOWN).

Алгоритм работы пульта включает следующие этапы:

1) Инициализация (setup): На этом этапе происходит настройка радиомодуля NRF24L01: установка адресов для передачи ("Transmitter") и приема ("Receiver"), активация динамической длины пакетов, установка низкого уровня мощности и запуск режима приема. Кроме того, здесь же происходит инициализация и настройка элементов управления. Устанавливаются пины энкодера в режим INPUT_PULLUP и сохранение начального состояния для отслеживания изменений. Настраиваются аналоговые пины джойстика в режим INPUT. А так же выставляются пины кнопок в режим INPUT_PULLUP для обнаружения нажатий.

2) Считывание данных (loop): При изменении состояния пина CLK энкодера определяется направление вращения (по состоянию пина DT) и обновляется значение тяги (remote_controls.gas). Значение ограничивается в диапазоне [-100, 100] с помощью функции constrain. Аналоговые сигналы джойстика с пинов VRX и VRY считываются, преобразуются в диапазон [-100, 100] с помощью функции map и инвертируются для соответствия направлению движения дрона. Проверяется состояние пинов кнопок UP_PIN и DOWN_PIN. При нажатии верхней кнопки формируется команда DEPART, нижней — LAND, в противном случае устанавливается NONE_COMMAND.

3) Отправка данных дрону (sendControlsToDrone): Радиомодуль переключается в режим передачи (stopListening), и структура remote_controls отправляется дрону через функцию radio.write. Если передача успешна, флаг receiver_online устанавливается в true, иначе — в false, сигнализируя о потере связи. После отправки радиомодуль возвращается в режим приема (startListening).

4) Прием телеметрии (readDroneState): При наличии данных в буфере радиомодуля (radio.available) считывается структура drone_state, содержащая текущую тягу и состояние дрона. Данные выводятся через последовательный порт для диагностики (printDroneState).

ЗАКЛЮЧЕНИЕ

Разработка беспилотного квадрокоптера с Х-образной архитектурой двигателей, описанная в данной работе, представляет собой комплексное исследование, охватывающее проектирование механической части, обработку сенсорных данных, управление ориентацией и организацию радиоуправления. Основной целью работы было создание прототипа полетного контроллера квадрокоптера с доступной аппаратной платформой на базе микроконтроллера Arduino Nano, способного выполнять стабильный и управляемый полет. В ходе работы были достигнуты значительные результаты, которые подтверждают успешность предложенного подхода и его применимость как в образовательных, так и в исследовательских целях.

На этапе проектирования и сборки механической части квадрокоптера была создана прочная и легкая конструкция на основе карбоновой рамы, оснащенная двигателями с рейтингом производительности 2750 кВ, регуляторами скорости оборотов и трехсекционным аккумулятором номинального напряжения 11.1В. Использование демпферов и тщательная установка компонентов в корпусе позволили минимизировать вибрации, обеспечив надежную основу для работы датчиков и алгоритмов управления. Разработанная система обработки данных с сенсоров с применением комплементарного фильтра обеспечила точное определение ориентации дрона с частотой обновления 100 Гц, что является достаточным для управления в реальном времени на ограниченных ресурсах Arduino Nano.

Управление ориентацией реализовано с использованием ПИД-регулятора, который эффективно компенсирует отклонения от заданных углов путем дифференциального распределения тяги между двигателями. Настройка коэффициентов ПИД-регулятора позволяет получить стабильный полет, несмотря на ограничения, связанные с шумом датчиков и дрейфом измерений. Система радиоуправления, основанная на радиомодуле NRF24L01, обеспечила надежную двустороннюю связь между пультом и дроном, позволяя оператору задавать тягу, ориентацию и специальные

команды (взлет, посадка) с частотой обновления 10 Гц. Программное обеспечение пульта, включающее обработку сигналов с джойстика, энкодера и кнопок, было оптимизировано для работы на Arduino Micro, демонстрируя устойчивость к временным потерям связи.

Достигнутые результаты подтверждают возможность создания функционального квадрокоптера с использованием недорогих и доступных компонентов. Прототип обладает достаточной стабильностью и управляемостью для выполнения базовых полетных задач, а разработанные алгоритмы и программное обеспечение могут быть адаптированы для других типов дронов или автономных систем. Теоретическая значимость работы заключается в исследовании и реализации комплементарного фильтра и ПИД-регулятора в условиях ограниченных вычислительных ресурсов, что вносит вклад в область управления робототехническими системами. Практическая значимость состоит в создании рабочего прототипа, который может служить основой для дальнейших разработок или использоваться в образовательных целях для изучения принципов управления беспилотными аппаратами.

Несмотря на достигнутые результаты, работа имеет потенциал и для дальнейшего совершенствования. Основные рекомендации и перспективы включают следующие направления:

- 1) Улучшение обработки данных сенсоров. Для повышения точности определения ориентации, особенно угла рысканья, рекомендуется интегрировать данные магнитометра с дополнительной фильтрацией для компенсации магнитных помех. Также возможно использование гибридных алгоритмов, сочетающих комплементарный фильтр с элементами фильтров Madgwick или Mahony, для повышения точности при минимальном увеличении вычислительной нагрузки.

- 2) Оптимизация ПИД-регулятора. Автоматизация настройки коэффициентов ПИД-регулятора с использованием методов, таких как

Ziegler-Nichols или генетические алгоритмы, позволит сократить время настройки и повысить стабильность управления. Добавление адаптивного ПИД-регулятора, изменяющего коэффициенты в зависимости от режима полета, может улучшить поведение дрона при резких маневрах.

3) Увеличение дальности и надежности связи. Для расширения дальности радиоуправления можно использовать радиомодули с более высокой мощностью передачи или альтернативные протоколы, такие как LoRa, обеспечивающие большую дальность при низком энергопотреблении. Также рекомендуется реализовать механизм автоматического переключения каналов для минимизации помех в диапазоне 2.4 ГГц.

4) Расширение функциональности. Добавление модуля GPS для навигации и автономного полета, а также камеры для передачи видеопотока, позволит расширить области применения дрона, включая аэрофотосъемку и мониторинг. Реализация алгоритмов автономного управления, таких как удержание высоты или следование маршруту, повысит уровень автоматизации системы.

5) Переход на более мощную платформу. Использование микроконтроллеров с большей вычислительной мощностью, таких как STM32 или ESP32, позволит реализовать более сложные алгоритмы управления и обработки данных, сохраняя компактность системы. Это также упростит интеграцию дополнительных сенсоров и модулей связи, но приведет к усложнению системы.

6) Тестирование в реальных условиях. Проведение полевых испытаний в различных условиях (ветер, перепады температур, электромагнитные помехи) позволит оценить надежность системы и выявить направления для дальнейшей оптимизации. Сравнение с коммерческими решениями, такими как Betaflight или Ardupilot, поможет определить конкурентные преимущества и недостатки разработанного прототипа.

Перспективы дальнейшей разработки включают создание модульной платформы, которая может быть адаптирована для различных задач, таких как доставка грузов, сельскохозяйственный мониторинг или спасательные операции. Также возможно использование разработанных алгоритмов в образовательных курсах по робототехнике и беспилотным системам, что способствует популяризации инженерных дисциплин. В целом, данная работа демонстрирует успешную реализацию базовой системы управления квадрокоптером и открывает широкие возможности для дальнейших исследований и практического применения.